Natural Language Processing

LLM-Augmented customer support triage and Quality Monitoring System

Aryaman Singh Dev
Christian Jay St Francis Clarke

School of Graduate Professional Studies

Artificial Intelligence

AI-574: Natural Language Processing
Spring 2026

Document Control

Work carried out by:

Name	Email Address	Other
Aryaman Singh Dev	**Asd5520@psu.edu**	
Christian Clarke	**Cjc6362@psu.edu**	

Revision Sheet

Release No.	Date	Revision Description
1	**Mar 22**	Project description
2	Mar 29	Literature survey and summary
3		
4		

TABLE OF CONTENTS

1 INTRODUCTION 3
2 PROBLEM STATEMENT 3
3 CHALLENGES 3
3.1 RELATED WORKS 3
3.2 IMPORTANCE AND IMPACTS 3
4 DATA COLLECTION 3
4.1 Evaluation Dataset 4
4.2 Adaptation Dataset 5
5 DATA PREPROCESSING 5
6 METHODOLOGY 6
6.1 Performance Benchmark 6
6.2 Fine-Tuning 7
6.3 Retrieval Augmented Generation 8
7 RESULTS AND INTERPRETATION 8
7.1 Performance Benchmark 8
7.2 Fine-Tuning 10
7.3 Retrieval Augmented Generation 12
8 DISCUSSION OF RESULTS 12
8.1 Implications 12
8.2 Limitations 12
8.3 Future Work 13
9 Your Feedback 13
10 REFERENCES 14
11 Appendix 14

1 INTRODUCTION

Customer support systems today must handle an overwhelming volume of unstructured text data, including emails, chat logs, support tickets, and customer feedback. Effectively processing this information is critical for organizations aiming to deliver timely, accurate, and high-quality service. From a Natural Language Processing (NLP) perspective, this challenge involves multiple interconnected tasks such as intent classification, entity extraction, sentiment and emotion detection, urgency identification, and summarization of complex, multi-turn conversations. Traditional rule-based systems and keyword-matching approaches often fail to capture the nuance and variability of human language, especially in cases involving ambiguity, multiple intents, or informal expressions.

To address these limitations, this project proposes an LLM-augmented, end-to-end NLP system designed to enhance customer support triage and quality monitoring. By leveraging advanced deep learning models and transformer-based architectures, the system aims to automatically classify and route tickets, extract key information, assess customer sentiment, generate concise summaries, and evaluate the quality of agent responses. Unlike many existing solutions that focus on isolated tasks, this project integrates multiple NLP components into a unified and transparent pipeline. The goal is to transform raw customer interactions into structured, actionable insights that improve operational efficiency, support decision-making, and enhance the overall customer experience.

2 PROBLEM STATEMENT

Modern customer support operations generate large volumes of unstructured text (emails, chat logs, tickets, and survey comments) that must be quickly understood, categorized, prioritized, and evaluated for quality. From an NLP perspective, this is a complex multi-task problem involving intent classification (what the customer needs), entity extraction (order IDs, product names, account details), sentiment/emotion detection (frustration vs satisfaction), urgency detection, and summarization of multi-turn dialogues into concise ticket overviews. Traditional rule-based systems and simple keyword search struggle with ambiguity, multi-intent messages, and evolving language, making neural NLP and LLM-based methods necessary to transform raw text into structured labels and scores that downstream systems can use for routing, escalation, and coaching agents.

3 CHALLENGES

Ambiguity and Multi-Intent Language where customers often describe multiple issues in a single message ("I need a refund and also want to change my address"), which makes single-label intent classification insufficient and motivates multi-label or hierarchical approaches. Informal language, typos, emojis, and code-switching increase the difficulty of robust intent and sentiment detection. Further, data quality, class imbalance, and labeling cost are issues to highlight. Real support data is noisy, and some categories (e.g., rare technical issues) will be heavily under-represented, leading to class imbalance and unstable performance on minority intents. High-quality labels for fine-grained tasks (e.g., detailed quality scores) are expensive to obtain, so the project may need to combine weak supervision, existing CSAT scores, and small manually labeled subsets. Issue on subjectivity in Quality Evaluation and Sentiment requires solving the problems below. Assessing "helpfulness," "empathy," or "tone" is inherently subjective, meaning that both humans and LLMs can disagree on what constitutes a "good" response. The system must be designed so that LLM-based quality scores are treated as suggestions and evaluated against human or CSAT ground truth rather than trusted blindly.

Evaluation of summaries and LLM judgements remains an important scenario to question. Automatic summarization metrics like ROUGE and BLEU only weakly correlate with human judgments for conversational data, so small-scale human evaluation will be important. LLM-generated quality scores and summaries require careful evaluation protocols to ensure usefulness and reliability.

Privacy, ethics and bias are an issue. Support logs may contain sensitive personal or financial information, raising privacy and anonymization concerns; the project will work with public or synthetic data and discuss anonymization strategies for real-world deployment.
Models for sentiment, urgency, and quality could be biased against certain language styles or demographics, so fairness and explainability (e.g., checking which words drive predictions) are important considerations.

3.1 RELATED WORKS

Customer support systems have increasingly incorporated machine learning and natural language processing (NLP) techniques to improve efficiency, scalability, and quality of service. Recent research has explored multiple components of customer-support automation, including intent classification, request routing, summarization, sentiment analysis, and quality monitoring. This literature review examines prior work across these areas and highlights key findings relevant to LLM-augmented customer support systems. There are existing research and industry solutions that address parts of this problem, but few open, end-to-end systems that integrate triage, summarization, and quality monitoring in a transparent way.
Research and academic work
Many papers tackle intent classification and intent mining for customer support, using classical machine learning, transformers, semi-supervised learning, and multi-task setups.

Dong et al. (2021) proposed a semi-supervised multi-task learning approach for customer intent classification. Their study addressed the challenge of incomplete and noisy labels in real-world datasets by leveraging unlabeled data and domain-adaptive pretraining. Using ALBERT as the base model, they achieved nearly a 20-point improvement in AUC-ROC compared to baseline models. This work demonstrates the importance of incorporating unlabeled data and realistic data conditions when designing customer-support classifiers. A key finding was that multi-task, and semi-supervised signals improve

robustness, especially on low-resource intents. This showcases the need to include baseline supervised classifier, along with a semi-supervised variant for minority intent classes. [1]

Chatterjee and Sengupta (2020) explored intent mining from historical conversations using unsupervised and weakly supervised learning techniques, thereby presenting context-aware conversation modeling. By leveraging the Universal Sentence Encoder and a clustering algorithm called ITER-DBSCAN, the authors identified hidden intent structures in conversation logs. Their results showed improvements in clustering performance compared to traditional methods. We learnt how to add an intent discovery offline stage on ticket history before finalizing label schema. This work emphasizes the value of discovering latent intent categories before building supervised models. [2]

Similarly, Arias-Barahona et al. (2023) focused on classifying technical support requests using traditional machine learning methods. Their study compared multiple algorithms and found that Support Vector Machines (SVM) achieved the highest accuracy of 98.97% after preprocessing and class balancing. This research highlights that classical machine learning approaches remain competitive, especially in structured and well-labeled datasets. [3]

Song et al. (2019) investigated customer satisfaction analysis using context-assisted multiple instance learning (CAMIL). Their model incorporated both sentiment and reasoning cues from dialogues and achieved strong performance in predicting customer satisfaction. This study demonstrates that analyzing conversation context, rather than isolated messages, leads to better understanding of customer experiences.
Sandbank et al. (2018) focused on detecting egregious conversations between customers and virtual agents. Using a feature-based SVM classifier, they identified problematic interactions with improved generalization across domains. Their findings indicate that interaction-level features, such as repeated failures or escalation patterns, are critical for effective quality monitoring. [4]

Guibon et al. (2021) examined emotion recognition in customer-service conversations using few-shot learning techniques. Their proposed ProtoSeq model incorporated sequential context and demonstrated improvements over baseline models, although overall performance remained limited due to data scarcity and class imbalance. This work highlights the challenges associated with emotion detection in real-world customer interactions. [5]

Feigenblat et al. (2021) introduced the TWEETSUMM dataset, a benchmark for customer-service dialog summarization. Their study compared extractive and abstractive summarization approaches, demonstrating that fine-tuned neural models such as BART significantly outperform traditional methods, and evaluation with ROUGE and human checks. Their findings highlight the importance of domain-specific training when summarizing customer-service interactions. This exhibits dialog summarization is harder than generic summarization; context and speaker turn matter. We compare extractive baseline (TextRank) and abstractive model with human evaluation rubric, highlighting human-vs-automatic metrics gaps. [6]

Narotamo et al. (2023) scaled intent detection using targeted fine-tuning over relevant intent sets. They implemented transfer learning, intent subset selection, and large-scale fine-tuning. Compared precision, recall, F1 and scalability behavior across many intents to observe that careful intent selection and tuning outperform naïve full-taxonomy approaches. The paper depicts how to build a two-stage route, course intent family first, then fine-grained classifier. [7]

Rana et al. (2025) depicted explicit multi-intent detection for support utterances using multi-label modeling, thresholding and decision calibration. Using multi-label metrics like micro/macro F1, subset accuracy, Hamming-style analysis. The paper finds single-label assumptions miss frequent compound issues in real support text indicating that refund and address change cases should be handled by multi-label input schema. [8]

Finally, Mendonça et al. (2023) introduced the MAIA dataset for dialogue quality and emotion annotation in customer support. Their results showed that current models struggle with accurate quality prediction, emphasizing the complexity of evaluating customer-service interactions. The study suggests that human evaluation remains essential for validating automated systems. [9]

In conclusion, the literature indicates that while individual components of customer-support automation are well studied, integrating them into a unified system remains a challenge. Effective systems must combine classification, summarization, sentiment analysis, and quality monitoring while leveraging both supervised and unsupervised techniques. Additionally, context-aware models and human evaluation play a critical role in ensuring system reliability and effectiveness.

Commercial platforms like Zendesk, Freshworks, and IBM Watson Assistant use NLP for ticket routing, chatbots, and basic sentiment analysis, and industry guides explicitly highlight intent recognition, entity extraction, and sentiment as core components of AI-driven customer service. Recent generative-AI products and case studies describe LLM-based systems that summarize and score customer conversations (e.g., sales or support calls) to support quality assurance and coaching. However, most available solutions are either proprietary black boxes or focus on a single subtask (e.g., intent classification or chatbots or summarization), and rarely provide an open, end-to-end pipeline that connects ticket triage, conversation summarization, and explainable quality scoring with clearly defined evaluation metrics. This project aims to fill that gap at a course scale by implementing and evaluating a transparent NLP pipeline—from raw support text to structured triage labels, summaries, and quality scores—that can serve as a reproducible academic and portfolio piece.

5. DATASET

Datasets: sources, features, and preprocessing Aligning public corpora with the tasks in the LLM-augmented support system (triage, multi-intent, sentiment, emotion, summarization, quality proxies).

1) Customer support on Twitter (dialogue, noise, threads)

Field	Detail
Source	Customer Support on Twitter (ThoughtVector)
Size	Full corpus ~3M tweets; project subset 80k–100k messages (stratified sample)
Key features	Tweet text, timestamps, author IDs, response_tweet_id / conversation linkage
Tasks	Intent/topic signals, sentiment, multi-turn context, dialogue-style summarization, robustness to informal language
Notes	Channel differs from private tickets; use for generalization, not as sole source of routing labels

2) TWEETSUMM (dialogue summarization benchmark)

Field	Detail

Source	Paper and links: TWEETSUMM (arXiv)
Key features	Customer-service dialogues and reference summaries
Tasks	Primary summarization evaluation for support-style threads (vs generic news)
Metrics	ROUGE + small human rubric samples (automatic metrics are weak for dialogue)

3) News summary (auxiliary abstractive pairs)

Field	Detail
Source	News Summary
Key features	Article–headline (or article–summary) pairs
Tasks	Optional auxiliary pretraining / transfer for abstractive compression only
Limitation	Not a substitute for dialog summarization metrics; do not claim CS-thread performance from this alone

4) Amazon reviews (satisfaction proxy)

Field	Detail
Source	Amazon Reviews for NLP
Key features	Review text, star rating
Tasks	Weak proxy for text -> satisfaction; sanity-check lexical patterns vs numeric ratings
Notes	Not agent–customer dialogue; use with LLM-as-judge / ticket CSAT only as approximate validation

5) Single-table ticket dump (proposal alignment)

Field	Detail
Source	Customer Support Data
Size	>85000 entries
Key features	Issue Metadata, Categorization, Agent Information, Customer Feedback, Transactional Data
Tasks	Supervised triage, routing, class-imbalance analysis, predictive satisfaction

6) MAIA-DQE (customer-support emotion + dialogue quality)

Field	Detail
Source	Paper: Dialogue Quality and Emotion Annotations for Customer Support Conversations (GEM 2023); data & code: johndmendonca/MAIA-DQE

Size	Curated bilingual customer-support dialogues with rich labels (612 dialogues for ~25k sentences)
Key features	Multi-turn agent vs customer turns (inbound / outbound); sentence-level Emotion, Engagement, Correctness, Templated; turn-level Understanding, Sensibleness, Politeness, IQ; dialogue-level task success / dropped-conversation flags
Tasks	Fine-grained emotion models aligned with support; dialogue quality estimation that pairs directly with this project's LLM quality-monitoring story
Notes	Stronger fit than generic social text (e.g. Reddit): same domain as triage/summarization

Feature categories (cross-corpus)

Features are combined across Twitter, TWEETSUMM, news summary, Amazon, ticket CSV, and MAIA-DQE JSON.

Category	Description	Typical sources
Textual	Customer message, agent/brand reply, full thread or ticket body, dialogue turns (text_src / text_mt in MAIA), article body (news), review text (Amazon)	Twitter, TWEETSUMM, tickets, MAIA, news, Amazon
Labels	Issue type / category, multi-intent, emotion (MAIA sentence-level + triage sentiment axis), priority (when mapped from tickets), resolution / outcome, summarization targets (TWEETSUMM references)	Tickets, MAIA, TWEETSUMM, LLM-derived labels for Twitter subsets
Numeric	Scalar sentiment (triage), satisfaction / CSAT-style scores (tickets), star ratings (Amazon as proxy), quality rubric scores (MAIA; comparable spirit to /quality checks)	Tickets, Amazon, MAIA, API outputs
Metadata	Timestamps, IDs (tweet, ticket, dialogue), channel (public Twitter vs ticket system), floor direction (MAIA inbound/outbound), language (MAIA bilingual)	Twitter, tickets, MAIA

Preprocessing pipeline (shared)

i. Cleaning

- Unicode normalization, consistent whitespace.

- Twitter: strip/normalize URLs, handles, and noisy tokens; optional language filter if you restrict to English.

- Kaggle CSVs (tickets, Amazon, news): parse encodings safely; strip HTML or markup if present in text columns.

- MAIA-DQE: load JSON subsets from repo data/; align sentence lists with parallel annotation lists (same length per turn).

ii. Truncation

- Enforce max token/character length for transformer encoders and for LLM context (triage, summarize, quality).

- Long threads: chunk with overlap or summarize intermediate segments before full-thread modeling.

iii. Missing values

- Drop rows with empty primary text fields for supervised tasks; or impute labels only when a documented rule applies (e.g. unknown category).

- Document every rule per task (triage vs emotion vs summarization).

iv. Anonymization

- Hash or remove user IDs, ticket IDs in published artifacts; redact order/account patterns where required for demos or submission.

- Keep internal stable IDs for joining labels inside your train/val/test pipeline only.

v. Imbalance

- Class weights, stratified sampling, or resampling for rare ticket categories and rare emotions in MAIA.

- Optional synthetic augmentation for minority classes (document ethics and data lineage if LLM-generated).

vi. Splits & leakage

- Tickets / Amazon / news: stratified 70% / 15% / 15% where labels permit.

- Twitter / TWEETSUMM / MAIA: split by conversation or dialogue ID so turns from the same thread do not appear in both train and test.

6. METHODOLOGY

Description

This project proposes an end-to-end neural NLP system that augments customer support triage and quality monitoring using a combination of deep learning models and Large Language Models (LLMs).
Core components

Ticket Classification & Routing

Use models such as BiLSTM/CNN with word embeddings and transformer models (e.g., BERT/RoBERTa) to perform multi-label intent and urgency classification for each ticket (billing issue, login problem, refund request, bug report, feature suggestion, etc.).
The output will drive automatic routing to the appropriate queue or specialist.
Named Entity Recognition (optional but planned)
Apply sequence labeling models to extract key entities such as order numbers, product names, and dates from ticket text, reducing manual lookup effort for agents.

Sentiment and Emotion Detection

Build or fine-tune a sentiment/emotion classifier to estimate customer frustration or satisfaction level, which can be used to flag highly negative or at-risk interactions for faster handling.

Ticket Summarization

Implement summarization of multi-turn conversations into short, agent-friendly summaries capturing the core problem, attempted resolutions, and current status.

Compare an extractive baseline (e.g., TextRank) to an abstractive approach (e.g., T5 or prompting an LLM), especially drawing on dialog-style summarization work for customer service.
AI-Based Quality Monitoring

Design an LLM-based scoring module that evaluates agent responses along dimensions such as helpfulness, empathy, clarity, and resolution, producing a quality score per interaction.
Compare these scores to existing signals such as CSAT ratings where available, and analyze agreement/disagreement patterns.

Data Augmentation and Robustness

Use LLMs to generate synthetic examples of rare or edge-case tickets (e.g., unusual refund scenarios), improving robustness of intent and sentiment models, especially for under-represented classes.

Dataset

Primary dataset: Kaggle "Customer_support_data" or similar, which contains roughly 80k–90k historical customer support interactions, including free-text messages, categories, and customer satisfaction scores.

The team will perform standard splits (e.g., 70/15/15 train/validation/test), with proper stratification over major intent categories.

6) The project is not a classic trained BERT end-to-end for every task codebase.
It is inference-first: most deep behavior comes from calling pre-trained models, not from you running backprop on large custom heads inside the repo.

6.1 Implemented inference stack (what the code does today)

Layer	Mechanism	Notes
Primary models	Pre-trained LLMs via API (LLMClient)	Inference only- no gradient updates in this repo. Triage, quality, summarization use structured JSON prompts + validation
Dense retrieval	Sentence-transformers when RAG_BACKEND=embedding (RAGService)	Inference only on a fixed encoder for policy similarity
Deep Learning	Fine-tuned BERT and RoBERTa for category only (train_triage_transformer.py)	off by default; if enabled, adds a hint into the triage prompt – the LLM still decides the full structured output.
Classical baseline/hybrid	TF-IDF + logistic regression (scripts/train_encoder_classifier.py) + TriageService	Not a deep neural network, serves baseline metrics and a fast category hint
Lexical RAG (default)	Token overlap	No neural model

Not implemented in-repo: training T5 / BART for summarization; BiLSTM / CNN baselines; multi-task encoder + multiple heads; GPU training loops (AdamW, early stopping on your checkpoints); BERT token-classification NER for order IDs.

- Why not a full DL training monolith:
  labels, MLOps, time-to-ship; one LLM interface covers many behaviors.
- When full fine-tuning would be justified:
  cost/latency at scale, no cloud LLM, stable taxonomy + data, or explicit coursework to show Trainer + curves.

<u>ASSIGNMENT 4</u>

1. Deep Learning Process

This project implements an **LLM-augmented customer support pipeline** using a deployment-first architecture. The system is exposed through a FastAPI service with task-specific endpoints for triage, quality monitoring, summarization, and retrieval support.

At runtime, incoming support text is processed by modular services:

- **Triage service** predicts priority, category, sentiment, and route assignment.
- **Quality service** evaluates agent replies against a rubric.
- **Summarization service** compresses multi-turn support threads.
- **RAG service** optionally retrieves policy snippets (lexical or embedding-based) to ground outputs.

The primary deep-learning behavior is produced via **pre-trained LLM inference**. Outputs are constrained to structured JSON and validated by schema/domain rules to reduce response drift and improve reliability. This provides rapid end-to-end functionality without requiring large in-house labeled corpora for each subtask.

In addition to this core path, the project includes optional supervised components:

1. **Classical baseline** (TF-IDF + Logistic Regression) for category hints and benchmarking.
2. **Optional transformer fine-tuning** (BERT/RoBERTa) for single-task category classification, used as an alternate method to demonstrate model training and compare against prompting-based inference.

This design balances practical deployment goals with academic rigor: it supports production-like operation while still enabling explicit model-training experiments.

2. Choices of Deep Neural Networks and Justification

2.1 Primary model family: pre-trained LLMs (inference)

The core system relies on a configurable LLM backend for triage, quality, and summarization. This choice is justified by:

- **Multi-task capability** from one interface.
- Strong performance in low-label settings.
- Faster development and iteration compared with training multiple specialized models.
- Easier extensibility for prompt/rubric updates.

2.2 Dense retrieval: sentence-transformer embeddings

For policy grounding, the project supports embedding-based retrieval. This improves semantic matching beyond token overlap and is useful when lexical RAG misses paraphrased policy intent.

2.3 Encoder fine-tuning: BERT/RoBERTa

To satisfy training-centric evaluation requirements, the repository includes a Hugging Face training path for category classification. Encoder models (e.g., RoBERTa-base) are suitable because:

- They are strong for short-text classification tasks.
- They provide a clear supervised learning artifact (checkpoint + metrics).
- They are appropriate for controlled comparisons against baseline/classical models.

2.4 Why not train every architecture (e.g., T5/BART + multi-task + NER) in this milestone

Training and serving multiple deep models simultaneously increases:

- Data labeling demands,
- MLOps and compute complexity,
- integration overhead and debugging risk.

Given project scope, the chosen architecture focuses on high-impact functionality first, then adds optional training components where academically useful.

ASSIGNMENT 5

3. Training and Testing Strategy

3.1 Data and Split Strategy

To ensure fair and reproducible evaluation across all tasks in the LLM-augmented customer support system, each task uses a clearly defined labeled dataset and a task-specific split strategy. Data leakage is prevented by maintaining conversation-level separation for dialogue-based datasets and by applying stratified sampling where applicable.

Triage / Routing

The primary dataset for triage and routing is the **Customer Support Data** dataset, which contains approximately **85,000+ historical customer support interactions**. Labels include issue category, priority, and route assignment.

The dataset is split into:

- **70% Training**
- **15% Validation**
- **15% Testing**

A **stratified sampling** approach is applied based on issue category and priority labels to preserve class distribution across all partitions.

To address class imbalance before splitting:

- Minority classes are identified through label-frequency analysis.
- **Oversampling** is used for underrepresented classes.
- **Undersampling** is optionally used for dominant classes.
- **Class-weighted loss functions** are applied during supervised model training.
- **Synthetic augmentation** using LLM-generated rare support cases may be introduced for low-frequency intents.

After balancing, stratified splitting preserves the adjusted class distribution.

Sentiment / Emotion Detection

The sentiment and emotion detection task uses:

- **MAIA-DQE** dataset for dialogue-level emotion and quality annotations.
- **Amazon Reviews** dataset as a proxy for customer satisfaction and sentiment labels.

For MAIA-DQE, splitting is performed by **dialogue ID** to ensure turns from the same conversation do not appear in multiple sets.

For Amazon Reviews, splitting is performed by **review ID**.

The split ratio is:

- **70% Training**
- **15% Validation**

- **15% Testing**

Class imbalance for rare emotions is addressed using class weighting and oversampling techniques.

Summarization

The summarization task uses the **TWEETSUMM** dataset, which contains customer-service conversations paired with human-written summaries.

To prevent summary leakage, splitting is performed at the **conversation/thread level** rather than individual message level.

The split ratio is:

- **70% Training**
- **15% Validation**
- **15% Testing**

Auxiliary transfer learning may be performed using the **News Summary** dataset for abstractive summarization pretraining.

Quality Monitoring

The quality monitoring task uses:

- **MAIA-DQE** for dialogue quality and emotion annotations.
- **Customer Support Data** where CSAT or customer feedback exists.
- **Amazon Reviews** as an approximate proxy for satisfaction.

The split is performed by **conversation ID** or **ticket ID**.

The split ratio is:

- **70% Training**
- **15% Validation**
- **15% Testing**

This ensures that interactions from the same support thread do not appear across multiple partitions.

Conversation-Level Separation

For conversational datasets such as **Customer Support on Twitter**, **TWEETSUMM**, and **MAIA-DQE**, splitting is performed at the **conversation, thread, or dialogue ID level** rather than the message level. This prevents information leakage where turns from the same interaction could appear in both training and testing sets, artificially inflating performance.

A fixed random seed is used to ensure reproducibility across all experiments.

4. Evaluation Metrics

4.1 Triage Metrics

The triage and routing system is evaluated using multiple classification metrics to account for both overall performance and class imbalance.

Metrics include:

- **Category Accuracy** for overall issue-category prediction.
- **Priority Accuracy** for urgency-level prediction.
- **Precision** to measure correctness of predicted labels.
- **Recall** to measure coverage of actual labels.
- **F1-score** as the harmonic mean of precision and recall.

To better analyze class imbalance:

- **Micro-averaged Precision, Recall, and F1-score** are reported to measure overall performance across all samples.
- **Macro-averaged Precision, Recall, and F1-score** are reported to evaluate all classes equally and highlight minority-class performance.
- **Weighted F1-score** is reported to account for label-frequency imbalance.

To visualize model behavior:

- **Confusion Matrices** are generated to identify commonly confused categories such as billing, refund, login issues, bug reports, and feature requests.

To further evaluate minority-class performance:

- **Per-class Precision, Recall, and F1-score** are reported for all intent categories.
- Minority and rare classes such as escalations or technical issues are separately analyzed.

For multi-intent classification tasks, the following additional metrics are used:

- **Subset Accuracy**

- **Hamming Loss**
- **Jaccard Similarity**
- **Micro-F1**
- **Macro-F1**

Special attention is given to minority classes by highlighting recall and F1-score on rare support intents.

Expected Results

The triage classification system will be evaluated using accuracy, micro-F1, macro-F1, weighted F1, and per-class metrics. A confusion matrix will be generated to visualize common misclassifications.

REFERENCES

[1] Dong, L., Spencer, M., Biagi, A. (2021). A Semi-supervised Multi-task Learning Approach to Classify Customer Contact Intents. Association for Computational Linguistics. https://aclanthology.org/2021.ecnlp-1.7/
[2] Chatterjee, A., Sengupta, Shubhashis (2020). Intent Mining from past conversations for Conversational Agent.
https://aclanthology.org/2020.coling-main.366.pdf
[3] Barahona, A., Arteaga, H., Arias, O., Ruiz, J., Diaz, M., Soto, R. (2023). Requests classification in the customer service area for software companies using machine learning and natural language processing.
https://pmc.ncbi.nlm.nih.gov/articles/PMC10280277/
[4] Song, K., Bing, L., Gao, W., Lin, J., Zhao, L., Wang, J., Sun, C., Liu, X., Zhang, Q. (2019). Using Customer Service Dialogues for Satisfaction Analysis with Context-Assisted Multiple Instance Learning. EMNLP-IJCNLP.
https://aclanthology.org/D19-1019/
[5] Guibon, G., Labeau, M., Flamein, H., Lefeuvre, L., Clavel, C. (2021). Few-Shot Emotion Recognition in Conversation with Sequential Prototypical Networks. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 6858–6870, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics. https://aclanthology.org/2021.emnlp-main.549/
[6] Feigenblat, G., Gunasekara, C., Sznajder, B., Aaronov, R., Konopnicki, D., Joshi, S. (2021). TWEETSUMM – A Dialog Summarization Dataset for Customer Service. IBM Research AI. https://arxiv.org/pdf/2111.11894.pdf
[7] Narotamo, N., Aparicio, D., Mesquita, T., Almeida, M. (2023). Intent Detection at Scale: Tuning a Generic Model using Relevant Intents. Zendesk Arxiv.
https://arxiv.org/pdf/2309.08647.pdf
[8] Rana, D., Kharod, S., Kaushik, B., Sharma, S. (2025). Multi-Intent Detection in Customer Support Queries Using AI. 2025 International Conference on Engineering Innovations and Technologies (ICoEIT)
https://ieeexplore.ieee.org/document/11210877/
[9] Mendonca, J., Pereira, P., Menezes, M., Cabarrao, V., Farinha, A., Moniz, H., Lavie, A., Trancoso, I. (2023). Dialogue Quality and Emotion Annotations for Customer Support Conversations. Association for Computational Linguistics.
https://aclanthology.org/2023.gem-1.2/

General Guidelines

1. Please use this template document to complete each deliverable assignment.
2. Each assignment must be submitted by the due date in the Course Schedule.
3. All figures should be followed by a brief description of the figure.
4. Figures can be hand-drawn and scanned in some circumstances, but the hand-drawn figure should be clear and legible to obtain full credit. Unclear hand-drawn figures will receive partial credit. For constructing figures and diagrams it is advised to use tools.

5. Figures and tables should have appropriate captions. For documenting and referencing styles please follow the APA or MLA writing style.
6. Please make sure that you provide a reference section.
7. Any material text or figure taken from books, journals or the Internet should be referenced. If you have a sentence or a figure that does not belong (authorship) to you, they must be clearly referenced. If you fail to do so your report will be considered as a case for plagiarism. It is your duty to make sure that your report is free from any activity related to plagiarism. Please see the section on Academic Integrity found below. The penalty for plagiarism will be a "0" awarded to your report. Thus, it is good to keep it simple, always have the principle to acknowledge people for their contributions.
8. Please note that the use of ChatGPT is prohibited. If the use of ChatGPT to complete the assignment has been determined, then you will receive NO credit for your project.

Project Assignment 5 (10 points)

Provide the following details related to your project

- Deep Learning process
    - Training/Testing strategy
    - Evaluation metrics
- Students will be provided with some examples of projects completed in the past.

Project Assignment 6 (10 points)

Provide the following details related to your project

- Obtained results
- References/Appendix
- Conclusion of your study
- Appendix should include the codebase and instructions to run your application.
- Prepare a demo version of your application which will be presented in week 7

Project Assignment 7 (10 + 5 = 15 points)

Submit the final report after implementing all my feedback. Make sure that all the sections in the report are complete. In addition to that, the team will have an opportunity to present their project outcomes in the classroom for about 10-15 minutes. More instructions about the presentation will be provided by the instructor over the email.